

I used ChatGPT and Gemini as my two GenAIs, I used the same copy-pasted prompt for both AIs as follows. I used them through their respective websites directly. The prompt is long so as to include all the instructions, so please zoom in to read it.

i need to ask 2 genais the following so you're first
this will be in c++, note that the assignment says "generate c++ code using objects, not functions, to implement the following requirements"
• The program will prioritize emails for a busy company CEO.
• You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
• You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
• You must create functions from scratch. Do not include pre-existing heap modules.

the commands will be as follows:
EMAIL <sender category>, <subject line>, <date>
The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:
• Boss – read first
• Subordinate – read next
• Peer – read next
• ImportantPerson – read next
• OtherPerson – read last
If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.
EMAIL is followed by space. The rest of the fields are delimited.
Assume <sender category> is one of the five strings listed above.
Assume <subject line> is a string which may contain spaces, but not commas
Assume <date> is in the format: MM-DD-YYYY

NEXT
Next email for the CEO to read. Display the information on the terminal in the following format:
Sender:
Subject:
Date

READ
CEO has read email and dealt with it

COUNT
Display current count of unread files

The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

the following sample test file
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT

should have the following output
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read

Gemini produced the following program: (Multiple screenshots, as requested)

```

1  #include <iostream>
2  #include <string>
3  #include <sstream>
4
5  // Custom object to represent individual Emails
6  class Email {
7  private:
8      std::string sender;
9      std::string subject;
10     std::string date; // Format: MM-DD-YYYY
11
12     // Convert sender category string to integer priority rank (higher = higher priority)
13     int getSenderPriority() const {
14         if (sender == "Boss") return 5;
15         if (sender == "Subordinate") return 4;
16         if (sender == "Peer") return 3;
17         if (sender == "ImportantPerson") return 2;
18         if (sender == "OtherPerson") return 1;
19         return 0;
20     }
21
22     // Convert MM-DD-YYYY into YYYYMMDD string for simple Lexicographical comparison
23     std::string getFormattedDate() const {
24         if (date.length() != 10) return date;
25         std::string mm = date.substr(0, 2);
26         std::string dd = date.substr(3, 2);
27         std::string yyyy = date.substr(6, 4);
28         return yyyy + mm + dd;
29     }
30
31 public:
32     Email() : sender(""), subject(""), date("") {}
33
34     Email(const std::string& s, const std::string& sub, const std::string& d)
35         : sender(s), subject(sub), date(d) {}
36
37     std::string getSender() const { return sender; }
38     std::string getSubject() const { return subject; }
39     std::string getDate() const { return date; }
40
41     // Comparator overLoad for MaxHeap priority ordering:
42     // 1. Primary: Sender Priority Rank (Higher wins)
43     // 2. Secondary: Date (Newer wins)
44     bool operator<(const Email& other) const {
45         int p1 = this->getSenderPriority();
46         int p2 = other.getSenderPriority();
47
48         if (p1 != p2) {
49             return p1 < p2;
50         }
51         return this->getFormattedDate() < other.getFormattedDate();
52     }
53
54     bool operator>(const Email& other) const {
55         return other < *this;
56     }
57 };
58
59 // Node structure for List-based heap
60 struct Node {
61     Email data;
62     Node* next;
63     Node(const Email& email) : data(email), next(nullptr) {}
64 };
65
66 // Linked-List implementation of a MaxHeap
67 class MaxHeap {
68 private:
69     Node* head;
70     int heapSize;
71
72     // Helper to get element at index (0-indexed)

```

```

72 // helper to get element at index (0 indexed)
73 Node* getNodeAt(int index) {
74     Node* curr = head;
75     for (int i = 0; i < index && curr != nullptr; ++i) {
76         curr = curr->next;
77     }
78     return curr;
79 }
80
81 void swapNodes(Node* a, Node* b) {
82     Email temp = a->data;
83     a->data = b->data;
84     b->data = temp;
85 }
86
87 void heapifyUp(int index) {
88     while (index > 0) {
89         int parentIndex = (index - 1) / 2;
90         Node* currNode = getNodeAt(index);
91         Node* parentNode = getNodeAt(parentIndex);
92
93         // If current is greater than parent, swap up
94         if (currNode->data > parentNode->data) {
95             swapNodes(currNode, parentNode);
96             index = parentIndex;
97         } else {
98             break;
99         }
100     }
101 }
102
103 void heapifyDown(int index) {
104     while (true) {
105         int leftChild = 2 * index + 1;
106         int rightChild = 2 * index + 2;
107         int largest = index;
108
109         Node* largestNode = getNodeAt(largest);
110
111         if (leftChild < heapSize) {
112             Node* leftNode = getNodeAt(leftChild);
113             if (leftNode->data > largestNode->data) {
114                 largest = leftChild;
115                 largestNode = leftNode;
116             }
117         }
118
119         if (rightChild < heapSize) {
120             Node* rightNode = getNodeAt(rightChild);
121             if (rightNode->data > largestNode->data) {
122                 largest = rightChild;
123                 largestNode = rightNode;
124             }
125         }
126
127         if (largest != index) {
128             Node* currNode = getNodeAt(index);
129             swapNodes(currNode, largestNode);
130             index = largest;
131         } else {
132             break;
133         }
134     }
135 }
136
137 public:
138     MaxHeap() : head(nullptr), heapSize(0) {}
139
140     ~MaxHeap() {
141         while (head != nullptr) {

```

```

142         Node* temp = head;
143         head = head->next;
144         delete temp;
145     }
146 }
147
148 int getSize() const {
149     return heapSize;
150 }
151
152 bool isEmpty() const {
153     return heapSize == 0;
154 }
155
156 void insert(const Email& email) {
157     Node* newNode = new Node(email);
158     if (head == nullptr) {
159         head = newNode;
160     } else {
161         Node* Last = getNodeAt(heapSize - 1);
162         Last->next = newNode;
163     }
164     heapSize++;
165     heapifyUp(heapSize - 1);
166 }
167
168 Email peek() const {
169     if (isEmpty()) {
170         return Email();
171     }
172     return head->data;
173 }
174
175 void pop() {
176     if (isEmpty()) return;
177
178     if (heapSize == 1) {
179         delete head;
180         head = nullptr;
181         heapSize = 0;
182         return;
183     }
184
185     Node* LastNodePrev = getNodeAt(heapSize - 2);
186     Node* LastNode = LastNodePrev->next;
187
188     // Move Last node's data to root
189     head->data = LastNode->data;
190
191     // Detach and delete Last node
192     LastNodePrev->next = nullptr;
193     delete LastNode;
194     heapSize--;
195
196     heapifyDown(0);
197 }
198 };
199
200 // Object representing the CEO's Inbox Handler
201 class CEOInboxManager {
202 private:
203     MaxHeap inboxHeap;
204
205 public:
206     void processCommand(const std::string& Line) {
207         if (Line.empty()) return;
208
209         if (Line.rfind("EMAIL", 0) == 0) {

```

```

210         // Remove "EMAIL " prefix
211         std::string data = Line.substr(6);
212         std::stringstream ss(data);
213
214         std::string sender, subject, date;
215         std::getline(ss, sender, ',');
216         std::getline(ss, subject, ',');
217         std::getline(ss, date, ',');
218
219         // Trim Leading spaces if present
220         if (!sender.empty() && sender[0] == ' ') sender = sender.substr(1);
221         if (!subject.empty() && subject[0] == ' ') subject = subject.substr(1);
222         if (!date.empty() && date[0] == ' ') date = date.substr(1);
223
224         inboxHeap.insert(Email(sender, subject, date));
225     }
226     else if (Line == "COUNT") {
227         std::cout << "There are " << inboxHeap.getSize() << " emails to read.\n";
228     }
229     else if (Line == "NEXT") {
230         if (inboxHeap.isEmpty()) {
231             Email top = inboxHeap.peek();
232             std::cout << "Next email:\n";
233             std::cout << "Sender: " << top.getSender() << "\n";
234             std::cout << "Subject: " << top.getSubject() << "\n";
235             std::cout << "Date: " << top.getDate() << "\n";
236         }
237     }
238     else if (Line == "READ") {
239         if (inboxHeap.isEmpty()) {
240             inboxHeap.pop();
241         }
242     }
243 }
244 };
245
246 int main() {
247     CEOInboxManager manager;
248     std::string Line;
249
250     while (std::getline(std::cin, Line)) {
251         manager.processCommand(Line);
252     }
253
254     return 0;
255 }

```

ChatGPT produced the following program: (Multiple screenshots)

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <sstream>
5
6  using namespace std;
7
8  // Email object
9  class Email
10 {
11 private:
12     string sender;
13     string subject;
14     string date;
15     int priority;
16
17 public:
18     Email()
19     {
20         sender = "";
21         subject = "";
22         date = "";
23         priority = 0;
24     }
25
26     Email(string s, string sub, string d)
27     {
28         sender = s;
29         subject = sub;
30         date = d;
31
32         if (sender == "Boss")
33             priority = 5;
34         else if (sender == "Subordinate")
35             priority = 4;
36         else if (sender == "Peer")
37             priority = 3;
38         else if (sender == "ImportantPerson")
39             priority = 2;
40         else
41             priority = 1;
42     }
43
44     string getSender()
45     {
46         return sender;
47     }
48
49     string getSubject()
50     {
51         return subject;
52     }
53
54     string getDate()
55     {
56         return date;
57     }
58
59     int getPriority()
60     {
61         return priority;
62     }
63
64     // Returns true if this email should be ahead of the other email
65     bool higherPriorityThan(Email other)
66     {
67         // Sender category has priority first
68         if (priority != other.priority)
69             return priority > other.priority;
70
71         // If sender category is the same, newer date comes first.
72         // MM-DD-YYYY is converted into YYYYMMDD so it can be compared.

```

```

72 // MM-DD-YYYY is converted into YYYYMMDD so it can be compared.
73 int thisDate = getDateValue();
74 int otherDate = other.getDateValue();
75
76 return thisDate > otherDate;
77 }
78
79 int getDateValue()
80 {
81     int month = stoi(date.substr(0, 2));
82     int day = stoi(date.substr(3, 2));
83     int year = stoi(date.substr(6, 4));
84
85     return year * 10000 + month * 100 + day;
86 }
87 };
88
89
90 // MaxHeap object
91 class MaxHeap
92 {
93 private:
94     vector<Email> heap;
95
96     // Move an element upward
97     void heapifyUp(int index)
98     {
99         while (index > 0)
100         {
101             int parent = (index - 1) / 2;
102
103             if (heap[index].higherPriorityThan(heap[parent]))
104             {
105                 Email temp = heap[index];
106                 heap[index] = heap[parent];
107                 heap[parent] = temp;
108
109                 index = parent;
110             }
111             else
112             {
113                 break;
114             }
115         }
116     }
117
118     // Move an element downward
119     void heapifyDown(int index)
120     {
121         int size = heap.size();
122
123         while (true)
124         {
125             int left = 2 * index + 1;
126             int right = 2 * index + 2;
127             int Largest = index;
128
129             if (left < size &&
130                 heap[left].higherPriorityThan(heap[Largest]))
131             {
132                 Largest = left;
133             }
134
135             if (right < size &&
136                 heap[right].higherPriorityThan(heap[Largest]))
137             {
138                 Largest = right;
139             }
140

```

```

141         if (Largest != index)
142         {
143             Email temp = heap[index];
144             heap[index] = heap[Largest];
145             heap[Largest] = temp;
146
147             index = Largest;
148         }
149         else
150         {
151             break;
152         }
153     }
154 }
155
156 public:
157
158     // Add an email to the MaxHeap
159     void insert(Email email)
160     {
161         heap.push_back(email);
162         heapifyUp(heap.size() - 1);
163     }
164
165     // Look at the highest-priority email without removing it
166     Email peek()
167     {
168         return heap[0];
169     }
170
171     // Remove the highest-priority email
172     void removeMax()
173     {
174         if (heap.empty())
175             return;
176
177         heap[0] = heap[heap.size() - 1];
178         heap.pop_back();
179
180         if (!heap.empty())
181             heapifyDown(0);
182     }
183
184     // Return number of emails
185     int getSize()
186     {
187         return heap.size();
188     }
189
190     // Check if heap is empty
191     bool isEmpty()
192     {
193         return heap.empty();
194     }
195 };
196
197 // Main program object
198 class EmailManager
199 {
200 private:
201     MaxHeap inbox;
202
203 public:
204
205     // Add an email to the CEO's inbox
206     void addEmail(string sender, string subject, string date)
207     {
208         Email email(sender, subject, date);
209     }

```

```

209     Email email(sender, subject, date);
210     inbox.insert(email);
211 }
212
213 // Display the highest-priority email
214 void nextEmail()
215 {
216     if (inbox.isEmpty())
217         return;
218
219     Email email = inbox.peek();
220
221     cout << "Next email:" << endl;
222     cout << "Sender: " << email.getSender() << endl;
223     cout << "Subject: " << email.getSubject() << endl;
224     cout << "Date: " << email.getDate() << endl;
225 }
226
227 // Read and remove the highest-priority email
228 void readEmail()
229 {
230     if (inbox.isEmpty())
231         return;
232
233     inbox.removeMax();
234 }
235
236 // Display number of unread emails
237 void countEmails()
238 {
239     cout << "There are " << inbox.getSize()
240         << " emails to read." << endl;
241 }
242 };
243
244
245 int main()
246 {
247     EmailManager manager;
248
249     string line;
250
251     // Read commands until end of file
252     while (getline(cin, line))
253     {
254         if (line.substr(0, 5) == "EMAIL")
255         {
256             // Remove "EMAIL "
257             string emailData = line.substr(6);
258
259             // Find the commas separating the fields
260             int firstComma = emailData.find(',');
261             int secondComma = emailData.find(',', firstComma + 1);
262
263             string sender = emailData.substr(0, firstComma);
264
265             string subject = emailData.substr(
266                 firstComma + 1,
267                 secondComma - firstComma - 1
268             );
269
270             string date = emailData.substr(secondComma + 1);
271
272             manager.addEmail(sender, subject, date);
273         }
274     }

```

```

274         else if (Line == "NEXT")
275         {
276             manager.nextEmail();
277         }
278         else if (Line == "READ")
279         {
280             manager.readEmail();
281         }
282         else if (Line == "COUNT")
283         {
284             manager.countEmails();
285         }
286     }
287
288     return 0;
289 }

```

2E:

- ChatGPT
 - Correctness: The overall logic and priority rules are correct conceptually, but it uses a vector instead of the linked-list maxheap.
 - Execution time: Efficient, but it comes from the system using a vector instead of a linked-list
 - Space complexity: Since the system uses a vector and doesn't need a pointer for each email, it is efficient.
 - Maintainability: The system is relatively easy to understand because it separates Email, MaxHeap, and EmailManager classes, and its heap implementation is straightforward.
- Gemini
 - Correctness: The overall logic and priority rules are correct, and it follows the list-based heap implementation.
 - Execution time: Slower than ChatGPT since finding a parent/child requires walking through the list. On smaller lists it isn't a problem, but the issue grows as the emails increase.
 - Space complexity: Since it uses a linked-list, it requires slightly more memory to create a next pointer for each node
 - Maintainability: The code has a clear class structure, but the linked-list implementation would be more complicated to maintain.

2F: I would use Gemini's code because its data structure actually follows the assignment requirements. I think it would be easier to improve the existing linked-list system than to change ChatGPT's vector heap to a linked-list heap.

2G: I would improve Gemini's code by adding clearer comments.